

## ¿Qué es JSX?

**JSX** (*JavaScript XML*) es una extensión sintáctica para JavaScript desarrollada por Meta. Permite describir visualmente la interfaz de usuario escribiendo una sintaxis prácticamente idéntica a HTML de forma directa dentro del código de JavaScript. Aunque luce como HTML, no lo es; el compilador (como Babel o Vite/esbuild) transforma cada elemento JSX en llamadas a la función estándar `React.createElement(...)`.

## Características principales

- **Combinación de marcado y lógica:** Une la estructura visual y la lógica de renderizado en un mismo archivo (componente).
- **Inserción de expresiones JavaScript:** Permite evaluar cualquier expresión válida de JavaScript (variables, cálculos, ternarios, funciones) dentro de la plantilla envolviéndola entre llaves `{}`.
  - `<p>Estado: {activo ? "Activo" : "Inactivo"}</p>`
- **Elemento raíz único:** Toda expresión JSX debe retornar un único elemento padre contenedor (o bien un Fragmento de React: `<> ... </>`).
- **Propiedades en CamelCase:** Atributos estándar de HTML que coinciden con palabras reservadas de JavaScript se adaptan (por ejemplo, `class` se convierte en `className` y `for` en `htmlFor`).
- **Cierre estricto de etiquetas:** Todas las etiquetas deben cerrarse de forma explícita (por ejemplo: `<img />`, `<input />`, `<br />`).

## ¿Para qué sirve?

Sirve para simplificar la creación y estructuración de la interfaz de usuario en React, haciéndola más intuitiva, declarativa y fácil de leer y mantener, evitando la manipulación manual del DOM o la concatenación engorrosa de cadenas HTML.

```
import React from 'react';
function PerfilUsuario() {
  const nombre = "Carlos Mendoza";
  const activo = true;
  const materias = ["Ingeniería de Software II", "Bases de Datos",
"Programación Web"];
  return (
    <div className="card-usuario">
      <h2>Perfil del Estudiante: {nombre}</h2>
      <p>Estado: {activo ? "Activo" : "Inactivo"}</p>
      <h4>Materias Cursadas:</h4>
      <ul>
        {materias.map((materia, index) => (
          <li key={index}>{materia}</li>
        ))}
      </ul>
    </div>
  )
}
```

```
    </div>
  );
}
export default PerfilUsuario;
```

Explicación del ejemplo:

1. Dentro del bloque return se escribe la estructura visual con etiquetas similares a HTML (<div>, <h2>, <p>, <ul>, <li>).
2. Se utiliza la propiedad className en lugar de class para no colisionar con la palabra reservada de JavaScript.
3. Se integran expresiones de JavaScript entre llaves {}:
  - {nombre} imprime dinámicamente el valor de la variable.
  - {activo ? "Activo" : "Inactivo"} evalúa una condición lógica en tiempo real para determinar el texto visible.
  - {materias.map(...)} ejecuta un bucle iterativo que genera un elemento <li> para cada elemento del array.

## 2. Componentes en React

¿Qué son los componentes?

Los **componentes** son los bloques de construcción fundamentales e independientes de una aplicación en React. Representan piezas de la interfaz gráfica que encapsulan su propio marcado (JSX), estilos y lógica de funcionamiento. Conceptualmente funcionan como funciones de JavaScript que reciben entradas (*props*) y devuelven una descripción de lo que debe verse en la pantalla (elementos JSX).

Características principales

- **Modularidad y Reutilización:** Permiten fragmentar interfaces complejas en partes pequeñas y autosuficientes que pueden reutilizarse en distintas secciones del proyecto.
- **Componentes Funcionales:** En el desarrollo contemporáneo de React, se definen como funciones de JavaScript que retornan JSX.
- **Nomenclatura en PascalCase:** Sus nombres siempre inician con letra mayúscula (por ejemplo, TarjetaProducto, MenuNavegacion) para distinguirse de los elementos HTML nativos (div, button).
- **Mantenibilidad y Composición:** Pueden anidarse e interactuar entre sí (componentes padres que contienen componentes hijos), estructurando una jerarquía limpia tipo árbol.

¿Para qué sirven?

Sirven para evitar la repetición de código, dividir responsabilidades dentro de la aplicación, facilitar las pruebas unitarias y permitir el mantenimiento escalable de interfaces de usuario.

```

import React from 'react';
// Componente Hijo: BotonAccion
function BotonAccion() {
  return (
    <button className="btn-accion">
      Confirmar Operación
    </button>
  );
}
// Componente Padre: PanelControl
function PanelControl() {
  return (
    <div className="panel-contenedor">
      <h1>Panel de Administración</h1>
      <p>Seleccione una de las siguientes opciones:</p>
      <BotonAccion />
      <BotonAccion />
    </div>
  );
}
export default PanelControl;

```

Explicación del ejemplo:

1. Se define el componente BotonAccion, el cual encapsula únicamente la vista y estilo de un botón estándar.
2. Se define el componente PanelControl, que representa la vista principal.
3. Dentro de PanelControl se invoca a <BotonAccion/> dos veces. React renderiza ambas instancias de manera independiente, demostrando cómo una pieza aislada se reutiliza sin duplicar código.

### 3. Props en React

¿Qué son los props?

Las **props** (abreviatura de *properties*) son el mecanismo que utiliza React para transferir datos y configuraciones desde un componente padre hacia un componente hijo. Constituyen el canal principal de comunicación en el árbol de componentes.

Características principales

- **Inmutables (Solo lectura):** Un componente hijo nunca debe modificar sus propias props. Si los datos cambian, la actualización debe provenir del componente padre a través del estado.
- **Flujo unidireccional:** La información viaja estrictamente de arriba hacia abajo (de padre a hijo).

- **Flexibilidad de tipos:** Pueden pasarse como props cadenas de texto, números, booleanos, arreglos, objetos e incluso funciones (como *callbacks* de eventos) y otros componentes de React.
- **Recepción como objeto:** En los componentes funcionales, las props se reciben como un único argumento en forma de objeto (generalmente desestructurado como { titulo, precio }).

¿Para qué sirven?

Sirven para parametrizar componentes, permitiendo que un mismo componente visual muestre información diferente según los datos que reciba, logrando una interfaz dinámica y desacoplada.

```
import React from 'react';
// Componente Hijo que recibe props (desestructuración directa)
function TarjetaCurso({ titulo, profesor, duracionSemanas }) {
  return (
    <div className="tarjeta-curso">
      <h3>{titulo}</h3>
      <p><strong>Profesor:</strong> {profesor}</p>
      <p><strong>Duración:</strong> {duracionSemanas} semanas</p>
    </div>
  );
}
// Componente Padre que envía las props
function CatalogoCursos() {
  return (
    <div className="catalogo">
      <h2>Oferta Académica</h2>
      <TarjetaCurso
        titulo="Ingeniería del Software II"
        profesor="Lic. Martínez"
        duracionSemanas={16}
      />
      <TarjetaCurso
        titulo="Bases de Datos Relacionales"
        profesor="Ing. Gómez"
        duracionSemanas={12}
      />
    </div>
  );
}
export default CatalogoCursos;
```

Explicación del ejemplo:

1. TarjetaCurso define tres props: titulo, profesor y duracionSemanas.
2. El componente padre CatalogoCursos renderiza dos tarjetas pasando valores distintos a cada propiedad (las cadenas entre comillas y los números entre llaves).
3. El componente hijo toma esos valores e inyecta la información en los lugares correspondientes del JSX, manteniendo una misma estructura de diseño para contenidos diferentes.

#### 4. Firebase

## ¿Qué es Firebase?

**Firebase** es una plataforma integral de desarrollo de aplicaciones web y móviles (modelo *Backend-as-a-Service* o BaaS) desarrollada originalmente por Firebase Inc. y adquirida por Google en 2014. Proporciona a los desarrolladores un conjunto de herramientas en la nube que resuelven la infraestructura del backend (servidores, base de datos, autenticación, almacenamiento de archivos, hosting) para que el equipo pueda enfocarse en la interfaz y la lógica de la aplicación del lado del cliente.

### Características principales

- **Base de datos en tiempo real (Cloud Firestore y Realtime Database):** Bases de datos NoSQL basadas en documentos y colecciones que sincronizan los datos de manera instantánea entre todos los clientes conectados sin necesidad de realizar peticiones de sondeo manuales.
- **Autenticación gestionada (Firebase Authentication):** Sistema completo para gestionar altas, inicios de sesión y control de sesiones mediante correo y contraseña, números de teléfono o proveedores externos (Google, GitHub, Facebook, etc.).
- **Cloud Storage:** Almacenamiento seguro en la nube diseñado para alojar y servir contenido multimedia generado por los usuarios (como imágenes, audios o documentos en PDF).
- **Hosting rápido y seguro:** Alojamiento de producción para aplicaciones web estáticas o Single Page Applications (SPA como las construidas en React), con certificado SSL gratuito provisto automáticamente y despliegue por CDN global.
- **Cloud Functions:** Permite ejecutar código de backend (Node.js) en un entorno *serverless* disparado automáticamente por eventos de Firebase (como la creación de un nuevo usuario o una modificación en la base de datos).
- **Capacidades Offline:** Las bibliotecas de Firebase almacenan en caché local los datos consultados; si el dispositivo pierde la conexión a internet, la aplicación continúa funcionando y sincroniza los cambios en cuanto se restablece la red.